HEX SETTLERS

Engineering Diagnostic Reference

Backend JavaScript and interaction architecture - 6 November 2026

A code-level guide to the authoritative engine, browser/server interaction paths, mechanics, security boundaries, persistence, AI, diagnostics, and tests.

This is the code-oriented reference for maintaining Hex Settlers. It explains how browser interactions become protocol messages, how the JavaScript server validates and mutates authoritative state, how public/private views are built, when data is persisted, and which tests protect each subsystem. It contains no runtime user records, credentials, tokens, email addresses, chat, room snapshots, or private hands.

Reading convention: line references in source control change over time. Search by the function, message, action, or state-field names printed here.

Engineering index

1 System topology
2 Repository and runtime boundaries
3 Authoritative state model
4 End-to-end interaction pipeline
5 Protocol and transport
6 Authentication and accounts
7 Rooms, membership, and reconnect
8 Lobby rules and game start
9 Geometry, board generation, ports, and previews
10 Turn phases and timers
11 Authoritative action dispatcher
12 Settlement and city code
13 Road, ship, and route code
14 Production, bank, Gold, and dice
15 Robber, pirate, and stealing
16 Trading engine
17 Development cards
18 VP, titles, and completion
19 Scenario-specific implementation
20 Paired 5-6-player turns
21 Pause, undo, early end, departure, and rematch
22 Ranked matchmaking and rating
23 AI and neural training
24 Privacy projections, logs, and statistics
25 History, replay, and postgame
26 Textures, audio, and caching
27 Browser rendering and performance
28 Persistence and shutdown
29 Security model
30 Action and message catalog
31 Backend module catalog
32 Testing and change recipes
33 Current diagnostic findings

1. System topology

Hex Settlers is server-authoritative. The browser owns presentation, local preferences, input collection, canvas hit testing, prompts, and reconnect UX. The Node.js server owns identity, room membership, rule validation, random outcomes, hand/bank mutation, timers, AI, scoring, history, and persistence.

Layer	Primary files	Responsibility
Browser shell	`public/index.html`, `public/styles.css`	Controls, modal/panel structure, responsive presentation
Browser controller	`public/app.js`	DOM events, WebSocket client, canvas, prompts, audio, replay UI
Protocol	`server/protocol.js`, `src/shared/protocol.ts`	Allowed message vocabulary, field and size checks
Runtime entry	`src/server.ts` -> `dist/server.js` -> `server.js`	Production bootstrap and live engine
Authoritative engine	`server.js`	Rooms, maps, actions, timers, AI, projections, HTTP/WS, persistence
Rule modules	`server/*.js`	Testable policy for trades, dice, ships, security, ratings, etc.
Edge proxy	`cloudflare/worker.mjs`	Dynamic proxy, origin policy, health route, email bridge
Data	JSON under `DATA_DIR`	Users, history, active rooms, neural model, texture assets

The core invariant is simple: a highlighted client choice is only a preview. `applyActionCore(room, playerId, action)` rechecks role, phase, turn, costs, identifiers, board legality, mandatory prompts, and scenario restrictions. Only an accepted server mutation is broadcast.

2. Repository and runtime boundaries

`npm run build` compiles the TypeScript bootstrap. Production starts `dist/server.js`, which validates configuration and loads the established JavaScript engine. TypeScript is currently the boot/shared-vocabulary boundary, not a second rules engine.

The top of `server.js` imports policy modules and defines shared constants: resource kinds, build costs, piece limits, development-card names, persistence paths, transport limits, timers, and environment-derived settings. Browser text is not the rules source.

The `server/` modules accept plain objects and return calculations/validation results. `server.js` remains the orchestration layer that sequences mutation, log/stat recording, sound, replay, persistence, and broadcast. That side-effect order matters.

3. Authoritative state model

Room

A room has a code, host, ranked flag, rules, timestamps, player/spectator lists, runtime socket map, preview, texture metadata, AI settings, ballot/rematch fields, and optionally a game.

Sockets and disposable previews are deliberately absent from saved active-room snapshots. `activeRoomSnapshot()` creates recoverable data; `restoreRoomFromPersistence()` recreates runtime-only structures.

Game

`newEmptyGame(room)` creates the container. `startGame(room)` fills it.

Field	Meaning
`phase`	State-machine gate: setup, await-roll, actions, discard, thief, game-over, etc.
`currentPlayerId`	Normal action authority
`turnNumber`	Card eligibility, logs, stats, replay, AI identity
`rules`	Frozen normalized lobby rules
`geom`	Tiles, nodes, edges, adjacency, buildings, roads, ships, ports
`players`	Resources, cards, stats, army, VP, departure fields
`bank`, `devDeck`	Server-owned supplies
`special`	Mandatory Gold/discovery prompt
`discard`	Simultaneous discard requirements and completion
`trade`	Current offer, responses, revision, expiry
`timer`	Segment key, deadline, duration, pause data
`longestRoad`, `largestArmy`	Holder and strict-surpass benchmark
`stats`, `resourceSummary`	Analytics later filtered per viewer
`replay`	Initial frame and successful action patches

Tiles use axial coordinates. Nodes are hex intersections with at most one building. Edges join nodes and can hold one road or ship. Precomputed adjacency arrays make legality and production deterministic.

4. End-to-end interaction pipeline

Mutating gameplay action

1. A handler in `public/app.js` reads the latest state and selected target.
2. `sendGameAction(action)` packages `{type:'game_action', action, roomCode}`.
3. `send(obj)` checks socket state, attaches room scope, encodes JSON, and catches a close-during-send race.
4. The server parses the frame and validates it through `server/protocol.js`.
5. The `game_action` WebSocket branch verifies authenticated room binding.
6. `applyAction(room, playerId, action)` establishes log/replay context and calls `applyActionCore()`.
7. The matching action branch validates, mutates, records, advances phases, and returns `{ok:true}` or an error.
8. Success records replay, schedules active-room persistence, and calls `broadcastState(room)`.
9. Each browser receives a viewer-specific projection, replaces local state, reconciles prompts/timers, and schedules one paint.

Rejected actions do not mutate or broadcast speculative state. The requester receives the error and retains the last accepted snapshot.

Query-only paths

`query_build_options` and `query_ship_move_targets` calculate legal targets from authoritative state without mutation. The browser caches them only until the next state packet.

5. Protocol and transport

`server/protocol.js` rejects unknown message types, missing/malformed fields, bad action shapes, and oversized ordinary JSON before deep engine code. Texture publication has a separate larger allowance; binary assets use HTTP so large images do not block gameplay messages.

The HTTP server handles health, auth/account/admin APIs, texture routes, and static files. `safePath()` prevents traversal. `sendHttpJson()` applies consistent security headers.

WebSocket upgrades enforce origin policy. A socket authenticates by HTTP-only cookie or legacy token migration. `activateAuthenticatedSocket()` registers it. `bindSocketToRoomMember()` atomically replaces an older seat socket. `client_ping` / `server_pong` maintain liveness.

On the browser, `connect()` rejects callbacks from retired sockets, keeps one reconnect timer, uses increasing backoff, runs heartbeat checks, retries stalled handshakes, and defers safe room-scoped messages until rejoin is confirmed.

6. Authentication and accounts

`normalizeUsername()`, `cleanDisplayName()`, `validateUsername()`, and `validatePassword()` canonicalize input. `createUser()` creates salt/hash identity data. `verifyPassword()` performs asynchronous verification. `issueAuthToken()` returns a raw token once but persists only a hash. `authenticateByToken()` supports hashed records and legacy migration.

`safeUserPublic()` is the projection boundary: no password material, session tokens, recovery tokens, or private email state.

`handleAuthHttp()` covers session/logout, token migration, verification, registration, password-reset request/completion, and legacy compatibility. `handleAccountHttp()` covers friends, profile, name/password change, recovery email, and safe user discovery. `handleAdminHttp()` rechecks administrator authority on every endpoint.

Email helpers generate hashed one-time tokens and public links, then call `server/email-delivery.js`. Tests use capture delivery without external transmission.

7. Rooms, membership, and reconnect

Function	Responsibility
`createRoom()`	Host membership, defaults, preview, AI/ranked fields
`joinRoom()`	Code, capacity, role, and membership checks
`rejoinRoom()`	Restores existing identity instead of duplicating it
`roomSnapshot()`	Lobby-safe public room data
`broadcastRoom()`	Room projections to live members
`sendRoomStateToSocket()`	One member's room plus private game view
`removeRoomMember()`	Removal, host transfer, game normalization
`retirePlayerFromGame()`	Returns hand, marks departure, keeps pieces
`cleanupRooms()`	Expiry policy for inactive rooms

`recoverSocketRoomBinding()` reconnects an authenticated socket to its recoverable member.
`roomByCodeWithPersistence()` can restore a room from disk when it is not memory-resident.

8. Lobby rules and game start

The `set_rules` message requires the unranked lobby host. It normalizes map, scenario, timers, dice, bank, discard, VP, deck, Friendly Robber, exploration, and surprise-map values. Map-affecting changes invalidate the preview and un-ready humans.

`start_game` rechecks host authority, readiness, capacity, AI fill, and preview validity. `startGame(room)` then:

1. freezes normalized rules;
2. selects geometry and generator;
3. consumes a legal preview or generates the final map;
4. creates ports, robber/pirate, bank, deck, players, and pieces;
5. initializes draft/Test Builder/scenario state;
6. initializes stats, replay, and timer;
7. enters first setup placement;
8. broadcasts and persists.

Surprise-map rules intentionally prevent the visible preview from becoming the final board.

9. Geometry, board generation, ports, and previews

`buildGeometry()` creates standard-radius topology. `buildGeometryFromAxials()` creates irregular scenario frames. Specialized axial generators define Through the Desert, Cartographer, Six Islands, and expanded Fog layouts.

Each `generateBoard*` function assigns terrain and numbers. Shared helpers prevent invalid red-number adjacency and keep 6/8 off Gold in Cartographer/Scattered Tiles.

`startSeafarersRobberAndPirate()` chooses legal initial thief tiles. `finalizeGeneratedSeafarersPirate()` repairs missing/duplicate pirates and excludes hidden border sea and unrevealed fog.

`generatePorts()` selects non-adjacent shore edges. Regional helpers cover Six Islands balance, Through the Desert regions, and New Shores outer islands. `playerHasPort()` and `tradeRatioFor()` turn coastal ownership into 3:1 or 2:1 rates.

`mapPreviewKey()` hashes relevant settings. `ensurePreview()` reuses or regenerates. Test Builder edit messages validate host, phase, scenario, tile/number/port kind, shoreline, non-adjacency, and automatic port removal when repainting invalidates an edge.

10. Turn phases and timers

Principal phases are `cartographer-draft`, four setup settlement/route phases, `main-await-roll`, `main-actions`, `discard`, thief choice/move/steal phases, and `game-over`. Mandatory Gold choices live in `game.special` and block unrelated actions.

Timer ownership:

- `phaseDurationMs()` chooses configured duration.
- `timerSegmentKey()` identifies the exact turn/phase/prompt.
- `syncTimer()` creates a deadline only when the segment changes.
- `snapshotCurrentPhaseTimer()` and `setTimerForCurrentPhase()` support pause/subflows.
- `handleTimeout(room)` resolves the expired segment.

Timeouts choose legal fallback draft tiles, setup placements, Gold resources, discards, thief targets, steals, trade expiry, votes, and turn ending. They use the same rule helpers rather than trusting client state.

11. Authoritative action dispatcher

`applyActionCore()` first checks: active non-over game; valid kind; ranked restrictions; pause restrictions; active non-departed actor; current-turn ownership unless explicitly out-of-turn; and pending Gold prompts.

Action	Authoritative work
`cartographer_draft_place`	Inventory-backed tile placement and draft advancement
`place_settlement`	Setup/main placement, cost, connection, exploration awards
`place_road`	Setup/free/paid road and fog exploration
`place_ship`	Sea legality, pirate/fog rules, free/paid placement
`move_ship`	Open-end move and current-opportunity lock
`upgrade_city`	Own settlement replacement and payment
`buy_dev_card`	Payment, deck removal, private card creation
`play_dev_card`	Eligibility and effect routing
`choose_discovery`	Fog Gold choice
`choose_production_gold`	Queued production choices
`roll_dice`	Dice algorithm, production or 7 sequence
`discard_cards`	Exact required discard and bank return
`choose_thief`	Robber versus pirate
`move_robber`, `robber_steal`	Land block and random steal
`move_pirate`, `pirate_steal`	Sea block and random steal
`bank_trade`	Port-aware basket exchange
trade proposal/response/finalize	Player offer lifecycle
undo proposal/response	Previous-turn restoration vote
endgame proposal/response	Unanimous early-end vote
`end_turn`	Standard/paired next-player transition

Dry-run simulation rooms call the same validator but suppress broadcasts, sound, persistence, history, and training.

12. Settlement and city code

Settlement validation order:

1. visible board node;
2. land contact on sea boards;
3. valid setup/main phase;
4. remaining piece;
5. scenario starting-region rule;
6. `settlementDistanceOk()`;
7. main-game owned road/ship connection;
8. affordability;
9. payment through `payCostStats()`.

After placement the engine records build stats and sound, stores the setup-route anchor or calculates exploration. `computeLandIslands()` and `playerHasBuildingOnIsland()` prevent duplicate awards. Through the Desert tracks far-side/region claims separately.

City upgrade requires `main-actions`, the actor's settlement, an available city, and the exact cost. It replaces the building type, records spend/build, emits structure audio, recomputes VP, and checks victory.

13. Road, ship, and route code

Road placement checks visible vacant edge, phase, land/coastal contact, piece supply, setup anchor or network connection, and cost. Road Building uses a private remaining allowance shared with ships.

Ship placement requires Seafarers, a sea/coastal edge after hidden-border trimming, no pirate adjacency, and connection through the player's building/ships. Fog rejects an edge adjacent only to unrevealed fog.

`shipIsMovable()` requires ownership and an open route end. Movement also rejects a ship placed during the current action opportunity, validates the destination as if the origin were absent, enforces pirate/fog rules at both ends, and records the one-move flag.

`computeLongestRoadForPlayer()` graph-searches owned road/ship edges without reusing an edge. Opponent buildings stop traversal. `recomputeLongestRoad()` preserves the holder on ties and transfers only on strict surpass.

14. Production, bank, Gold, and dice

`ensureBank()` normalizes supply. `bankGive()` and `bankReceive()` are primitive mutations; higher helpers pair them with statistics.

`distributeResources(game, roll)` finds matching tiles, skips sea/desert/robber, computes settlement/city demand, records blocked opportunity separately, pays legal supply, and queues Gold recipients in `game.special`.

Gold is never a hand kind. Production choices validate amount and bank availability, pay standard resources, then advance to the next recipient or resume the prior phase/timer. `grantStartingResourcesForSecondSettlement()` intentionally excludes Gold.

Dice algorithms live in `server/dice-algorithms.js`: random, exact 36-pair balanced bag, probability correction, and third-repeat prevention. `roll_dice` requires await-roll, records statistics, broadcasts roller-tagged audio, then distributes or starts discard/thief processing.

`discardRequired()` implements “above limit, floor half.” Submitted discard maps must exactly match required count and the player's hand. `randomDiscardMap()` is the timeout/AI fallback.

15. Robber, pirate, and stealing

`startThiefChoice()` remembers source/player and routes Classic directly to robber or Seafarers to piece choice.

Movement checks a different legal land/sea tile, fog restrictions, and Friendly Robber policy from `server/friendly-robber.js`. Victims derive from adjacent buildings or ships. The steal action confirms eligibility and non-empty hand. `randomResourceFrom()` selects the card server-side; the browser never chooses the stolen kind.

`server/private-log.js` gives resource detail only to thief/victim. `sendPlayerSfx()` targets “stolen from” audio to affected players, while movement audio may be room-wide.

16. Trading engine

Bank-trade support in `server/bank-trade.js` plus `tradeRatioFor()` enforces an independent 4:1, 3:1, or 2:1 ratio for every outgoing resource; matching total exchange groups; no same kind on both sides; sufficient hand; and sufficient bank. Legacy one-kind actions normalize to baskets. Success records deltas and can add five seconds to the live initiator segment.

`propose_trade` validates action phase, paired policy, positive non-overlapping sides, and initiator hand. It creates/replaces a trade ID and response map. `respond_trade` is allowed out of turn. Closing/reopening a popup does not erase response. Revision invalidates earlier relevance. `finalize_trade` rechecks both hands at execution, transfers atomically, records stats/log/audio, and adds ten seconds.

17. Development cards

`buildDevDeck()` creates the selected 13/25/38 distribution, unique IDs, and shuffle. Purchase pays cost, pops the deck, stamps `boughtTurn`, keeps the card private, and records statistics.

Play validates ownership, unplayed state, purchase-turn rule, one non-VP-per-turn rule, phase, and payload.

- Knight increments army and starts thief movement.
- Road Building creates two shared road/ship placements.
- Invention validates exactly two standard choices and bank supply.
- Monopoly transfers every selected-resource card from active opponents, records victim losses, and sends targeted audio.
- Victory Point reveals/marks the card and checks victory immediately, including purchase turn.

18. VP, titles, and completion

`computeVP()` derives score from buildings, revealed VP cards, titles, exploration fields, and scenario bonuses. It does not trust a browser total.

`recomputeLargestArmy()` requires three Knights and strict surpass. Longest Road uses the same holder-preserving tie policy.

`checkWin()` sets game-over/winner/end time, finalizes stats, applies eligible ranked rating exactly once, persists aggregate user stats, sends targeted result audio, trains neural examples outside dry-run, persists replay/history, schedules room saving, and broadcasts final state.

19. Scenario-specific implementation

Scenario	Code-specific behavior
Four Islands	Specialized terrain generator plus standard island-component scoring
Six Islands	Expanded geometry, balanced 11 ports, paired turns
Through the Desert	Start predicates, Gold setup exclusion, far-side/region VP, regional ports
Fog Island	Hidden tile payload, edge reveal, discovery award, revealed-sea pirate
New Shores	Main-island setup, outer Gold, regional ports
Cartographer	Per-seat inventory, tile action, draft rotation, final number/port/thief assignment
Scattered Tiles	Random Cartographer generator with Gold red-number protection
Test Builder	Host tile/number/typed-port editors and final Seafarers validation

Fog uses `pickRandomUnrevealedFogTileOnEdge()`, `revealFogTile()`, `awardFogDiscovery()`, and `maybeExploreFogFromEdge()`. Successful road placement, ship placement, or ship move can invoke exploration.

Cartographer inventory comes from `cartographerDraftDefaultInventoryForPlayer()`. The action consumes a tile and advances. `finishCartographer4ManualDraft()` assigns numbers without Gold 6/8, places thieves/ports, and begins setup.

20. Paired 5-6-player turns

`gameUsesPairedTurns()` identifies expanded maps. After Player 1 ends, the third remaining player to the left gets an action-only Player 2 stage: no roll/production, no player trade, bank/port trade allowed. Then the next Player 1 begins.

The stage has a distinct timer segment and ship opportunity identity. Undo snapshots and timeouts understand both stages so they do not merge two action opportunities.

21. Pause, undo, early end, departure, and rematch

`pauseGame()` snapshots remaining time. `resumeGame()` restores a new deadline. The global action gate allows only designated meta-actions while paused.

`captureTurnUndoPoint()` stores start-of-turn state. Unranked propose/respond actions restore only after approval, and restored play remains paused until Play.

Early end requires remaining-human unanimity. AI-only opposition uses a five-second withdrawal window. `finalizeApprovedEndVote()` chooses a deterministic tiebreak winner and uses normal stats/history/audio/persistence finalization.

`retirePlayerFromGame()` and `server/departure.js` return the hand, mark departure, preserve pieces, and normalize pending trade/vote/phase. Title benchmarks remain until strictly surpassed.

`createRematchRoomFrom()` creates/reuses one room. The message transfers only the requester. Ranked rematch returns to ready without silently re-enqueueing.

22. Ranked matchmaking and rating

`enqueueRankedPlayer()` creates a queue record. `rankedRatingRange()` widens tolerance with wait. `selectRankedMatch()` chooses four. `launchRankedMatch()` creates the server-owned room and timed three-option ballot.

`resolveRankedScenarioVote()` handles revisions/abstention/ties and applies `rankedRulesForScenario()`. Ranked locks dice, timers, target, Friendly Robber, and map policy, and disables undo/early end.

`server/ranked-rating.js` calculates multiplayer expectation while preserving the point pool. Ratings start at 1000, exclude AI, require authoritative ranked completion, and carry an idempotence marker. `server/rating-policy.js` provides validated preview/rebuild with confirmation hash and backups.

23. AI and neural training

`handleAI(room)` inspects the same phases/special prompts as humans, creates normal action objects, and calls the same `applyAction()`.

Normal AI uses direct legal heuristics. Expert enumerates candidate builds/trades/cards/thief targets, clones state, applies candidates in a dry-run room, and scores resulting state with configurable weights. Neural adds a learned value over 17 features including production, build progress, Gold income, opponent leader progress, and blocked production.

`cloneGameForSimulation()` excludes replay trajectories and external side effects. Completion invokes `recordNeuralAiTrainingSnapshot()` and `trainNeuralAiFromFinishedGame()`. Training scripts checkpoint batches, validate full-target victories, and merge models weighted by completed games.

24. Privacy projections, logs, and statistics

`buildSanitizableState(game)` creates a neutral reusable base. `sanitizeStateFor(game, viewerId, baseState)` restores only authorized exact hand/cards, selected spectator perspective, and filtered statistics/logs. `stateForRoomSocket()` selects role. `broadcastState()` builds the base once and caches equivalent projections.

`pushLog()` creates public-safe structured entries. `server/private-log.js` changes steal text per viewer. Chat passes through `sanitizeChatText()`.

Statistics helpers record turns, rolls, builds, trades, cards, resource deltas, blocked production, thief movement, steals, discards, and ship moves. `filterStatsForViewer()` removes private dimensions. `server/resource-summary.js` distinguishes actual hand loss from robber-blocked opportunity.

25. History, replay, and postgame

`ensureReplay()` stores the initial frame. Successful actions call `recordReplayStep()`, which stores patches rather than recursively embedding full replay state. Browser functions `applyHistoryReplayPatch()` and `materializeHistoryReplayFrame()` reconstruct frames.

`makeGameHistoryEntry()` creates safe summaries, replay/log, map/ranked metadata, stats, and rating deltas. `packHistoryEntry()` and `unpackHistoryEntry()` handle storage. Postgame functions in `public/app.js` derive tabs, charts, cards, duration, piece counts, and perspective from the authoritative snapshot.

26. Textures, audio, and caching

Texture HTTP handlers validate manifest/file names, size, ownership, and storage keys. Uploads write temporary assets and atomically complete them. Unreferenced storage cleanup is guarded. `server/texture-packs.js` normalizes IDs and paths.

Actions emit sound names, not media. `broadcastSfx()` sends room effects; `sendPlayerSfx()` sends private effects; `sendGameResultSfx()` targets winner/loser. The client maps names to pooled HTML Audio and per-effect volume.

First-gesture audio priming now saves target volume, sets zero, plays, pauses/rewinds, and restores on success/failure, preventing the former robotic mixture.

`public/sw.js` best-effort precaches PNG/WAV, serves cache-first, removes older generations, and ignores dynamic HTML/API. HTML, client registration, and service-worker cache share `mobile-summary-v22`.

27. Browser rendering and performance

`render()` coalesces state/pointer/resize/image bursts into one `requestAnimationFrame`. It reads `getBoundingClientRect()` once, caches it for `drawBoard()`, and clears frame/cache in `finally` even if drawing throws.

`fitBoardToViewport()` uses visible tile bounds. Manual pan/zoom disables auto-fit until reset. Screen caches support hit testing. Timer UI uses a server-time offset but never advances official phases. Prompt deduplication keys preserve in-progress controls across repeated packets.

Mobile split ratio, panel scales, colorblind mode, and audio are presentation-only local preferences.

28. Persistence and shutdown

`server/persistence.js` supplies atomic replacement, queued writes, coalescing latest-state writes, and flush.

Store	Contents
`users.json`	Identity, hashes, recovery state, aggregate stats, friends
`game_history.json`	Completed safe history/replay
`active_rooms.json`	Recoverable rooms without sockets/previews
`neural_ai_model.json`	Validated learned parameters
texture directory	Published assets referenced by storage keys

`scheduleActiveRoomsSave()` coalesces rapid changes. `gracefulShutdown()` stops admission/timers, flushes stores, closes transports, and exits. Fatal signal/exception paths use the same bounded routine.

29. Security model

- Salted asynchronous password hashing; hashed session and one-time tokens.
- HTTP-only SameSite Strict cookies, Secure when trusted TLS.
- Proxy headers honored only under configured trust.
- Same-host/allowlisted WebSocket origins.
- Sliding-window auth rate limit.
- Size-limited JSON/binary readers.
- Normalized static and texture paths.
- Server reauthorization for admin operations.
- CSP self-only assets/scripts and same-origin framing; `SAMEORIGIN` supports legal modal.
- Viewer-specific state/history prevents unauthorized exact hands or private steals.

30. Action and message catalog

`game_action` kinds

`cartographer_draft_place`, `place_settlement`, `place_road`, `place_ship`, `move_ship`, `upgrade_city`, `buy_dev_card`, `play_dev_card`, `choose_discovery`, `choose_production_gold`, `roll_dice`, `discard_cards`, `choose_thief`, `move_robber`, `robber_steal`, `move_pirate`, `pirate_steal`, `bank_trade`, `propose_trade`, `respond_trade`, `finalize_trade`, `propose_turn_undo`, `respond_turn_undo`, `propose_endgame`, `respond_endgame`, `end_turn`.

WebSocket messages

- Account/history: `auth_register`, `auth_login`, `auth_token`, `auth_set_display_name`, `get_game_history`, `get_game_history_entry`, `get_player_leaderboard`.
- Room/lobby: `create_room`, `join_room`, `rejoin_room`, `leave_room`, spectator messages, leave-game messages, `kick_player`, `rematch_room`, `set_ready`, `set_rules`, `set_player_color`, AI settings, map/Test Builder edits, `chat`, `start_game`, `pause_game`.
- Ranked/assets: queue join/leave, scenario vote, texture select/publish/fetch.
- Query/transport: `client_ping`, build/ship queries, `get_state`, `game_action`.

Every room-scoped message is checked against the socket's bound member identity. A client-supplied code is not authority.

31. Backend module catalog

Module	Owned logic
`active-rooms.js`	Snapshot validity/normalization
`ai-tactics.js`	Robber, pirate, victim tactical scoring
`bank-trade.js`	Mixed basket and ratio validation
`departure.js`	Resource return/departure normalization
`dev-cards.js`	Played-card removal
`dice-algorithms.js`	Dice modes
`email-delivery.js`	Delivery transports/policy
`friendly-robber.js`	Protected thief targets
`game-rules.js`	Defaults, ranked maps, bank/scenario helpers
`http-auth.js`	Cookies, JSON reader, TLS trust
`land-masses.js`	Components and exploration
`neural-objective.js`	Training rewards
`persistence.js`	Atomic/queued/coalesced JSON
`pirate-rules.js`	Pirate occupancy
`player-trade.js`	Trade-side overlap
`port-placement.js`	Non-adjacent shoreline selection
`private-log.js`	Viewer-specific steal text
`protocol.js`	Message vocabulary/validation
`ranked-rating.js`	Multiplayer rating
`rating-policy.js`	Weight policy/rebuild
`replay.js`	Replay sanitization/patches
`resource-summary.js`	Attributed gains/losses
`road-building.js`	Shared free placement allowance
`runtime-config.js`	Environment/origin/headers
`security.js`	Hashing and rate limit
`ship-rules.js`	Current-opportunity ship lock
`test-builder-ports.js`	Custom port legality
`texture-packs.js`	IDs/manifests/paths
`trade-time.js`	Live-segment timer bonuses
`user-management.js`	Safe users/admin identity

32. Testing and change recipes

`npm run check` builds production, runs Node tests, and syntax-checks both large JavaScript entry points.

Area	Representative tests
Full live server	`server.integration.test.js`
Client network/render/audio	`client-networking`, `client-rendering`, `client-audio`
Trades	`bank-trade`, `player-trade`, `client-player-trade`, `trade-time`
Maps/ships/scenarios	`ship-rules`, `land-masses`, `pirate-rules`, `port-placement`
Ranked/rating	`ranked-*`, `rating-studio-v14`
Persistence/security	`persistence`, `security`, `http-auth`, `runtime-config`
Replay/mobile/postgame	`replay*`, `mobile-*`, `postgame-*`
AI/training	`ai-tactics`, `neural-*`

Add an action

Add protocol vocabulary; send client intent without local commitment; add an `applyActionCore` branch; validate role/turn/phase/IDs/cost/supply/scenario before mutation; use bank/stat/log helpers; recompute VP/titles; define timeout/AI paths; sanitize private fields; record replay; add unit, privacy, timeout, and integration tests.

Add a scenario

Define geometry/generator; normalize key; add preview/final generation and ports; add setup predicates/state; hook settlement/road/ship/thief/production points; set VP/ranked policy if desired; add browser selection/assets; simulate ports, pirate, numbers, and full-target completion.

Add private state

Never put it in the neutral broadcast base. Remove it in `buildSanitizableState()`, selectively restore in `sanitizeStateFor()`, filter history/log/stats, and test owner, opponent, spectator, and replay views.

Diagnose “needs refresh”

Check console, release IDs, service-worker cache, WebSocket auth/rejoin, stale-socket guards, server projection/broadcast, and stale prompt keys. Do not hide an authoritative synchronization defect with forced reloads.

33. Current diagnostic findings

Corrections made

1. Pooled sounds were audibly primed together on the first gesture. Priming is now silent with reliable volume restoration.
2. Client service-worker registration used an older release ID than HTML/cache. All now use `mobile-summary-v22`.
3. Frame-deny headers blocked the game's legal modal. Security now permits same-origin framing only.

Verification

- 264 tests passed, 0 failed, 0 skipped.
- Production TypeScript bootstrap and JavaScript syntax checks passed.
- Six-player integration measured local authoritative action-to-state maxima of 7.0-8.3 ms.
- Live browser first load, reload, current asset tags, and corrected Terms modal completed without console warnings/errors.
- The PDF was rendered and visually checked for clipping, overlap, broken tables, and legibility.

Engineering risk

`server.js` is large. Its advantage is one visible authoritative state machine; its cost is change coupling. Continue extracting pure calculation/validation modules while keeping orchestration and side-effect order explicit. Do not duplicate rules in the browser or distribute one mechanic across opaque event chains.

Public-data statement

This reference names files, functions, messages, state fields, and tests. It contains no persistence contents or runtime personal/user data.